Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Experiment 1
Implementation of a Single Layer Perceptron for Binary Classification

1. Objective

The objective of this experiment is to understand the concept of an artificial neuron and implement a Single Layer Perceptron from scratch for binary classification. Students will learn the perceptron learning algorithm, understand the importance of activation functions, visualize the learning process, and evaluate the classifier using a real-world dataset.

2. Background Theory

An Artificial Neuron is the fundamental building block of a neural network. It receives one or more input features, multiplies them by corresponding weights, adds a bias term, and applies an activation function to determine the output.

The Single Layer Perceptron is the earliest supervised learning algorithm capable of solving linearly separable binary classification problems.

Mathematical Formulation

Weighted Sum

$$z = \mathbf{w}^T\mathbf{x} + b$$

where

- $\mathbf{x}$: Input vector
- $\mathbf{w}$: Weight vector
- b : Bias
- z : Linear combination

Prediction

$$\hat{y} = f(z)$$

Weight Update Rule

$$\mathbf{w}_{new} = \mathbf{w}_{old} + \eta(y - \hat{y})\mathbf{x}$$

Bias Update

$$b_{new} = b_{old} + \eta(y - \hat{y})$$

where η denotes the learning rate.

3. Activation Functions

An activation function determines the output of a neuron based on the weighted sum of its inputs. It introduces non-linearity, enabling neural networks to learn complex patterns. Although the perceptron uses only the Step Activation Function, modern deep learning models employ differentiable activation functions for efficient optimization through backpropagation.

Step Activation Function

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

The Step Activation Function produces a binary output and is therefore suitable only for linearly separable binary classification problems.

Activation	**Output Range**	**Typical Applications**
Step	$\{0, 1\}$	Classical Perceptron
Sigmoid	$(0, 1)$	Binary Classification Output Layer
Tanh	$(-1, 1)$	Hidden Layers (Traditional Networks)
ReLU	$[0, \infty)$	Hidden Layers in CNNs and MLPs
Leaky ReLU	$(-\infty, \infty)$	Avoiding Dying ReLU
Softmax	$(0, 1)$	Multi-class Classification Output Layer

Note: This experiment uses only the Step Activation Function. The remaining activation functions will be studied in subsequent experiments involving Multilayer Perceptrons and Deep Neural Networks.

4. Dataset Description

Dataset: Banknote Authentication Dataset

Source: UCI Machine Learning Repository

Download Link:

`https://archive.ics.uci.edu/dataset/267/banknote+authentication`

Instances	1372
Features	4 Numerical Features
Classes	2
Missing Values	None
Task	Binary Classification

Feature Description

- Variance
- Skewness
- Curtosis
- Entropy

Target Class

- 0 – Authentic Banknote
- 1 – Forged Banknote

5. Experimental Procedure

Task 1: Dataset Exploration

- Load the dataset.
- Display the first five samples.
- Determine dataset dimensions.
- Identify missing values.
- Display descriptive statistics.

Expected Output
Dataset summary and statistical information.

Task 2: Exploratory Data Analysis
Generate

- Histograms
- Correlation Heatmap
- Scatter Plot
- Boxplots

Task 3: Data Preprocessing

- Normalize all numerical features.
- Split the dataset into Training (80%) and Testing (20%).

Task 4: Perceptron Implementation
Implement from scratch

- Weight Initialization
- Bias Initialization
- Step Activation Function
- Forward Propagation
- Perceptron Learning Rule

Task 5: Model Training
Display

- Epoch Number
- Misclassified Samples
- Updated Weights
- Updated Bias

Task 6: Model Evaluation
Compute

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

Additional Tasks

1. Compare the Step and Sigmoid activation functions. Discuss why the Step Activation Function is not used in modern deep learning models.
2. Compare the implementation with Scikit-learn's Perceptron.
3. Repeat the experiment using learning rates 0.001, 0.01 and 0.1.
4. Explain why a Single Layer Perceptron cannot solve the XOR problem.
5. Study the effect of feature normalization on model convergence.

Report Contents

The report includes: Objective, Background Theory, Activation Functions, Dataset Description, Experimental Procedure, Source Code, Experimental Results, Generated Plots with Interpretation, Performance Tables, Discussion, Conclusion, and References.

6. Source Code

The complete source code for this experiment – dataset exploration, EDA, preprocessing, the from-scratch perceptron implementation, model training, evaluation, and all additional tasks – is available on GitHub at:

`https://github.com/Lunarya-git/Deep-Learning-Experiments/tree/main/Experiment_1_single-layer-perceptron`

7. Experimental Results

Task 1: Dataset Exploration

The dataset was loaded successfully with no missing values in any of the 5 columns. Its dimensions and the first five samples are shown below.

Dataset Shape	1372 rows × 5 columns
Missing Values (all columns)	0

First Five Samples

Index	Variance	Skewness	Curtosis	Entropy	Target
0	3.6216	8.6661	-2.8073	-0.4470	0
1	4.5459	8.1674	-2.4586	-1.4621	0
2	3.8660	-2.6383	1.9242	0.1065	0
3	3.4566	9.5228	-4.0112	-3.5944	0
4	0.3292	-4.4552	4.5718	-0.9888	0

Descriptive Statistics

	Variance	Skewness	Curtosis	Entropy
mean	0.4337	1.9224	1.3976	-1.1917
std	2.8428	5.8690	4.3100	2.1010
min	-7.0421	-13.7731	-5.2861	-8.5482
25%	-1.7730	-1.7082	-1.5750	-2.4135
50%	0.4962	2.3197	0.6166	-0.5867
75%	2.8215	6.8146	3.1793	0.3948
max	6.8248	12.9516	17.9274	2.4495

All four features are continuous and on different scales (e.g. Skewness ranges roughly -14 to 13 while Entropy ranges roughly -8.5 to 2.5), which is why the normalization step is performed in Task 3.

Task 2: Exploratory Data Analysis

Histograms, a correlation heatmap, pairwise scatter plots, and boxplots were generated for all four features; these are presented with their interpretations in Section 8 (Generated Plots with Interpretation).

Task 3: Data Preprocessing

All four numerical features were normalized to the $[0, 1]$ range using Min-Max scaling, and the dataset was split into training and testing sets in an 80:20 ratio.

Training Samples	1097
Testing Samples	275
Scaler Used	MinMaxScaler

Tasks 4 & 5: Perceptron Implementation and Model Training

The perceptron was implemented from scratch with zero-initialized weights and bias, the Step Activation Function, and the standard perceptron learning rule, then trained for 50 epochs at the default learning rate ($\eta = 1$). A snapshot of the epoch-wise training log is shown below; the full log is summarized in Section 9 (Performance Tables).

Epoch	**Misclassified**	**W1**	**W2**	**W3**	**W4**	**Bias**
1	166	-6.83	-3.87	-5.32	1.53	8
2	88	-8.31	-6.15	-7.69	1.16	10
...	...	...	...	...	...	...
49	22	-18.53	-18.27	-19.56	0.38	24
50	20	-18.09	-17.96	-20.79	0.62	24

The training error fluctuates rather than decreasing monotonically to zero, which indicates that the two classes in this dataset are close but not perfectly linearly separable.

Task 6: Model Evaluation

The trained perceptron was evaluated on the held-out 275-sample test set.

Accuracy	0.9891
Precision	1.0000
Recall	0.9764
F1-score	0.9880

Confusion Matrix

	Predicted: Authentic	**Predicted: Forged**
Actual: Authentic	148	0
Actual: Forged	3	124

Precision of 1.0 means every note the model predicted as forged was actually forged (zero false positives); the 3 misclassifications were forged notes that the model missed (false negatives), which is why recall is very slightly below 1.

8. Generated Plots with Interpretation

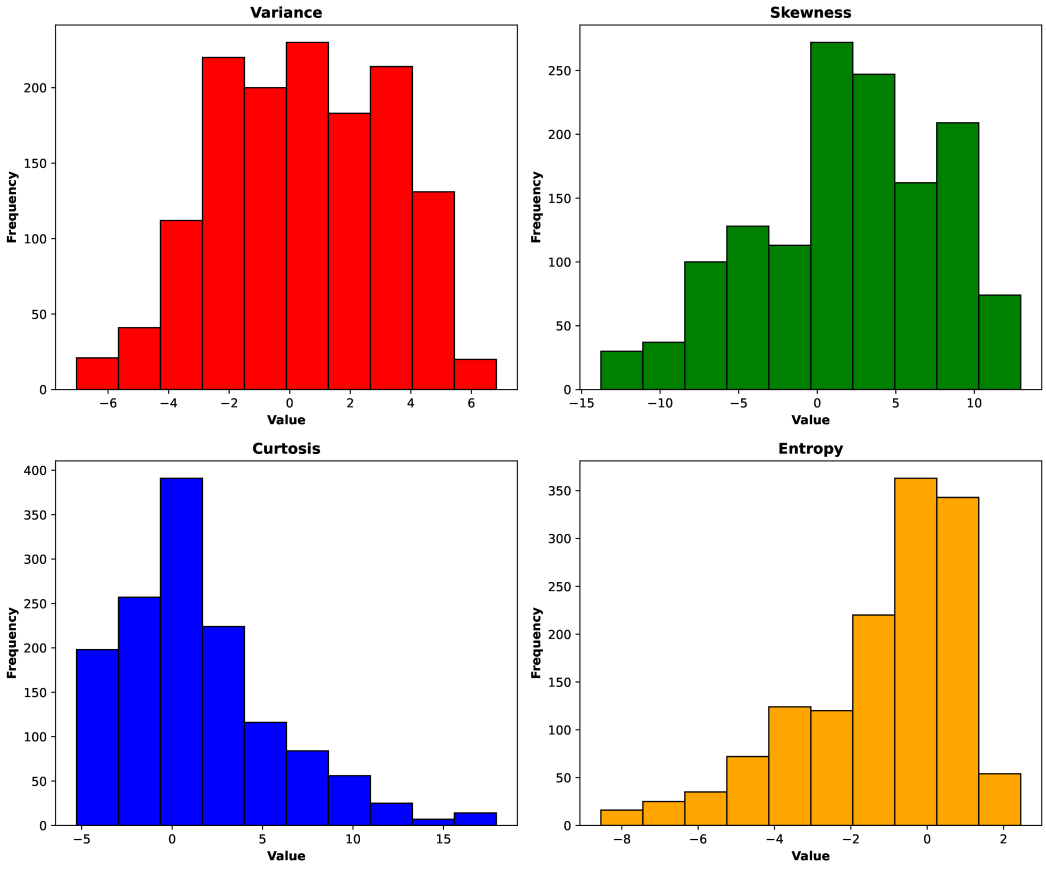

Figure 1: Distribution of the four input features.

Variance is roughly symmetric, indicating an even spread of data around the center, while Curtosis is right-skewed with extreme positive outliers, and both Skewness and Entropy are left-skewed, meaning their averages are pulled down by a few exceptionally low values.

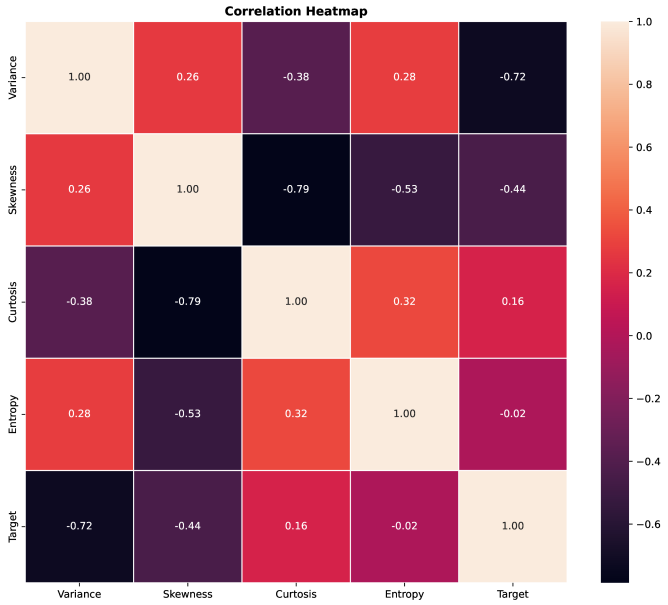

Figure 2: Correlation heatmap of the four input features.

Variance is strongly negatively correlated with Curtosis and Entropy, while Skewness and Curtosis are also negatively correlated with each other to some level. None of the correlations are extreme enough to determine a redundant feature that should be dropped.

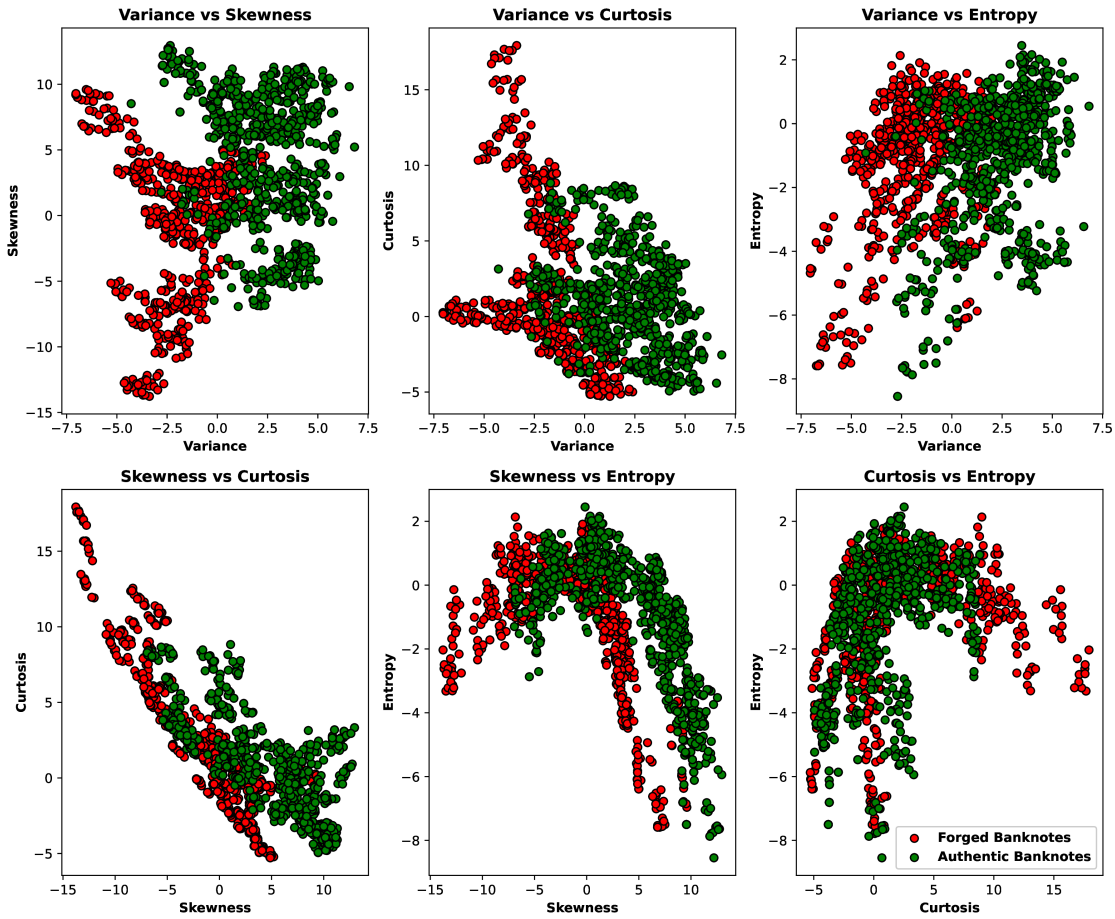

Figure 3: Pairwise scatter plots of the four features, coloured by class.

Variance vs. Skewness gives the cleanest separation possible between authentic and forged notes, these two features carry the most discriminative information for this task. Variance vs. Curtosis also shows some decent seperation. Some pairs (like Curtosis vs. Entropy) show much more overlap between the two classes, confirming the dataset is close to, but not linearly separable.

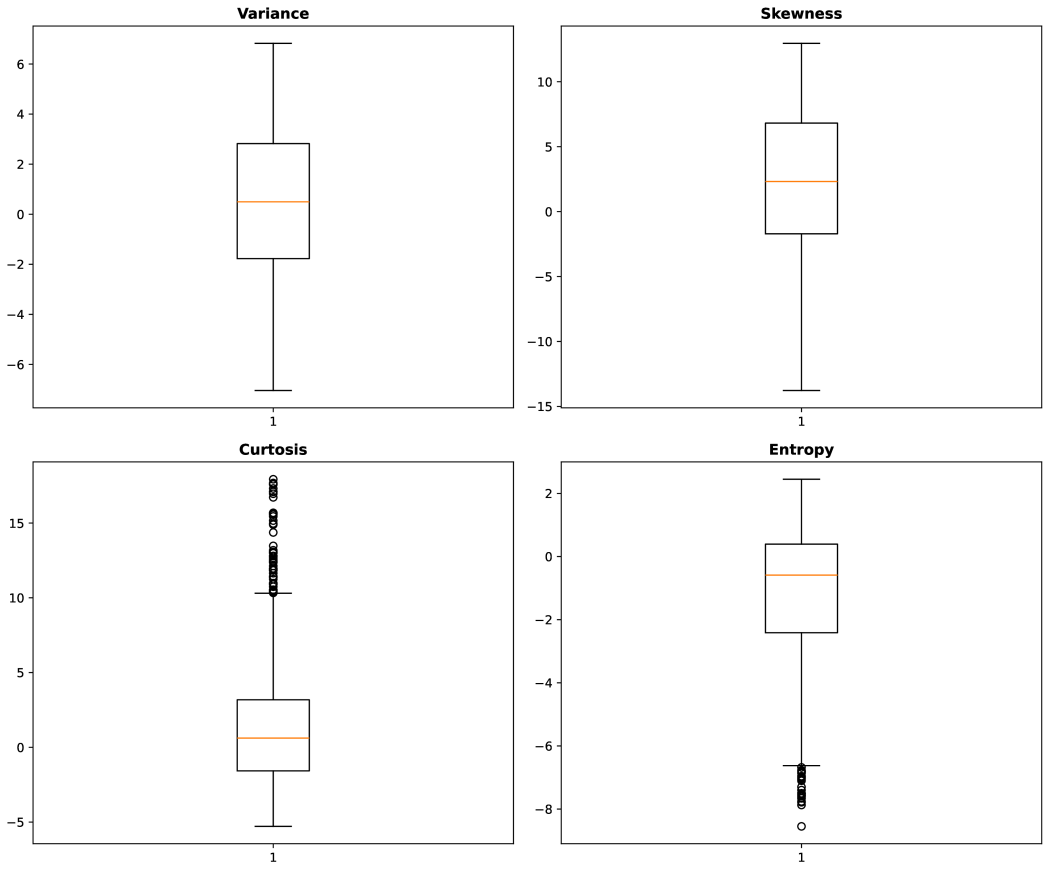

Figure 4: Boxplots of the four input features.

Curtosis and Skewness show the widest spread, and the most individual outlier points beyond the whiskers are shown by Curtosis and Entropy, matching the skew seen in their histograms. The Variance has the tightest and most symmetric spread of the four features.

Figure 5: Training error (misclassified samples) versus epoch.

The number of misclassified samples drops from 166 in epoch 1 to under 50 within the first 6

epochs, then continues to fluctuate in a slowly narrowing band down to 20 by epoch 50. It never reaches zero, which is the clearest evidence that the two classes are not perfectly linearly separable.

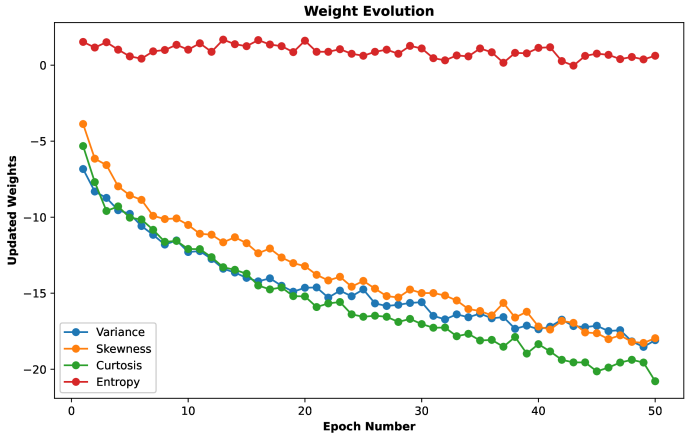

Figure 6: Evolution of the four feature weights across training epochs.

All four weights move steadily away from their zero initialization and roughly stabilize after about epoch 30, with the Curtosis weight ending up largest in magnitude. This matches the correlation heatmap and scatter plots, where Curtosis played a strong role in separating the two classes.

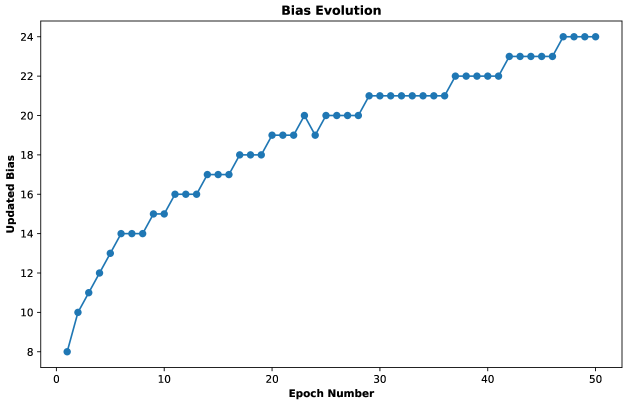

Figure 7: Evolution of the bias term across training epochs.

The bias increases steadily from 0 to 24 and then plateaus, shifting the decision boundary to better balance the two classes as training progresses. Its smooth, monotonic growth (unlike the noisier weight curves) suggests the bias update consistently pushes in the same direction for this dataset.

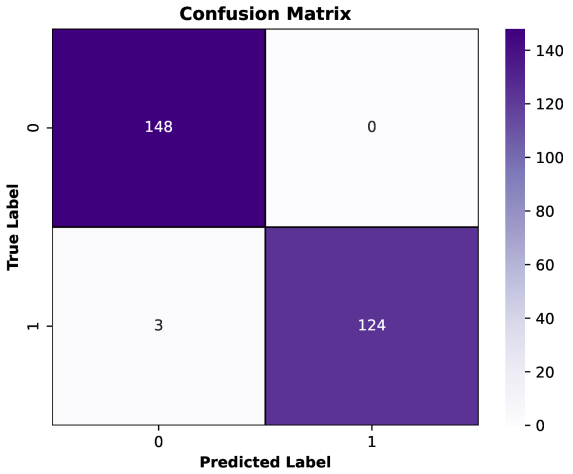

Figure 8: Confusion matrix on the test set.

The model made zero false positives and only 3 false negatives out of 275 test samples, so essentially all of its mistakes were forged notes mistakenly passed as authentic.

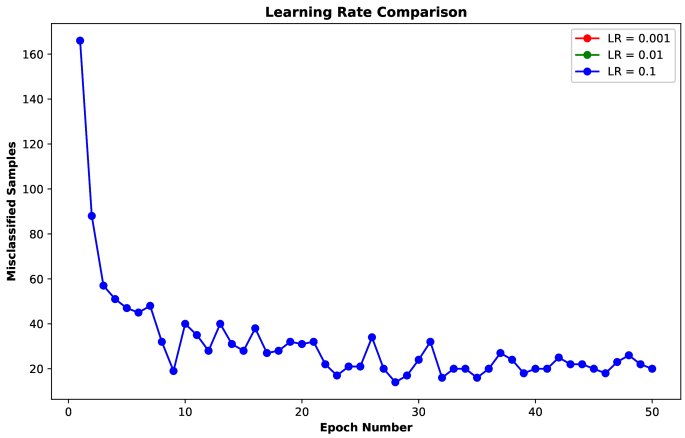

Figure 9: Training error versus epoch for learning rates 0.001, 0.01 and 0.1.

All three curves overlap perfectly. This alignment occurs because zero-initialized weights paired with a Step Activation Function rely solely on the *sign* of $\mathbf{w}^T\mathbf{x} + b$ to determine predictions. Modifying the positive learning rate scales $\mathbf{w}$ and b proportionally without altering their sign. Consequently, the sequence of misclassified samples remains identical regardless of η, affecting only the final weight magnitudes (see Discussion, Section 10).

9. Performance Tables

Training Summary

Dataset Size	1372 samples, 4 features
Train/Test Split	1097 / 275 (80% / 20%)
Learning Rate	1 (default run; 0.001 / 0.01 / 0.1 also studied – see Discussion)
Epochs	50
Final Weights	[-18.09, -17.96, -20.79, 0.62]
Final Bias	24
Accuracy	0.9891
Precision	1.0000
Recall	0.9764
F1-score	0.9880

Epoch-wise Learning

Epoch	Errors	Weight 1 (Variance)	Weight 2 (Skewness)	Bias
1	166	-6.83	-3.87	8
5	47	-9.78	-8.56	13
10	40	-12.29	-10.51	15
20	31	-14.64	-13.22	19
30	24	-15.59	-14.99	21
40	20	-17.37	-17.19	22
50	20	-18.09	-17.96	24

(Only Weight 1 and Weight 2 are shown to match the table format above; Weight 3 and Weight 4 follow the same pattern and are included in the full epoch-wise log in Section 7 and in the source code output.)

10. Discussion

Step vs. Sigmoid Activation

The Step function only outputs a hard 0 or 1 with a sharp jump at $z = 0$, so it is not differentiable at that point and has zero gradient everywhere else. This makes it unusable in backpropagation, since gradient-based optimizers need a smooth, non-zero gradient to update weights layer by layer. The Sigmoid function instead produces a smooth, continuous output between 0 and 1 with a well-defined derivative everywhere, which lets errors be propagated backward through many layers – this is why modern networks use differentiable activations (Sigmoid, Tanh, ReLU, etc.) instead of the Step function used in this experiment.

Comparison with Scikit-learn's Perceptron

Metric	Scratch Implementation	Scikit-learn Perceptron
Accuracy	0.9891	0.9636
Precision	1.0000	1.0000
Recall	0.9764	0.9213
F1-score	0.9880	0.9590

Both perceptrons implement the same underlying learning rule, so they perform comparably on this dataset. The scratch model updates on every misclassified sample every epoch, while scikit-learn's implementation adds internal optimizations (such as optional shuffling and early stopping), which is why the two end up with slightly different final weights and metrics – here the scratch implementation happened to edge out scikit-learn's on this particular split.

Effect of Different Learning Rates

As shown in the Learning Rate Comparison plot (Section 8), training with $\eta = 0.001$, 0.01, and 0.1 produced *identical* error-per-epoch curves. Because the weights start at zero and the Step Activation Function only responds to the sign of $\mathbf{w}^T\mathbf{x} + b$, uniformly scaling every update by a different positive learning rate never changes which samples are misclassified or in what order – it only rescales the final weight and bias magnitudes. In this specific from-scratch setup, the learning rate has no effect on the convergence curve itself, only on the numerical scale of the final parameters. (This is a property of a linear step-activated perceptron starting from zero weights; it would not generally hold for models with non-zero initialization or differentiable activations.)

Why a Single Layer Perceptron Cannot Solve XOR

XOR is not linearly separable: no single straight line can divide the four XOR points $\{(0,0) \to 0,\ (0,1) \to 1,\ (1,0) \to 1,\ (1,1) \to 0\}$ into two classes. A Single Layer Perceptron can only draw one linear decision boundary ($\mathbf{w}^T\mathbf{x} + b = 0$), so it can only solve problems where the classes can be separated by a straight line or hyperplane, like the banknote dataset used here. XOR needs at least one hidden layer (a Multilayer Perceptron) to combine two lines and carve out the correct non-linear decision regions.

Effect of Feature Normalization

	Without Normalization	With Normalization
Final Epoch Misclassified Samples	12	20
Test Accuracy	0.9818	0.9891

With a fixed learning rate, the raw (unnormalized) features have very different scales (e.g. Variance is roughly -7 to 7 while Entropy is roughly -8.5 to 2.5), so the geometry of the decision boundary the perceptron traces out is different from the normalized case – not simply a rescaled version of it, unlike the learning-rate case above. For this particular dataset, the raw-feature run happened to land on a slightly lower final training error within 50 epochs, but it generalized

marginally worse (lower test accuracy) than the normalized run. Normalization keeps weight updates proportionate across features and makes the training error curve more stable and easier to interpret in general.

11. K4-Question

Code the perceptron learning algorithm for OR, NOT, and AND gates using Python. Show the weights after each update. Plot the decision boundary after each weight update using Python built-in packages. (K4)

Perceptron Learning Rule

The same step-activated perceptron and update rule from Sections 2–3 were applied to the three elementary logic gates. Each gate uses binary inputs and a bias term; training proceeds sample-by-sample and the weight vector is updated only on misclassification, using

$$\mathbf{w}_{new} = \mathbf{w}_{old} + \eta(y - \hat{y})\mathbf{x}, \qquad b_{new} = b_{old} + \eta(y - \hat{y}).$$

Training was run with $\eta = 1$, zero-initialized weights, and the Step Activation Function. The tables below log every weight update in the order it occurred; the figures that follow plot the resulting decision boundary after each update.

AND Gate

Table 1: Weight updates during perceptron training for the AND gate.

Update #	Epoch	Input	Target	Prediction	Bias	w1	w2
1	1	(0, 0)	0	1	-1.0	0.0	0.0
2	1	(1, 1)	1	0	0.0	1.0	1.0
3	2	(0, 0)	0	1	-1.0	1.0	1.0
4	2	(0, 1)	0	1	-2.0	1.0	0.0
5	2	(1, 1)	1	0	-1.0	2.0	1.0
6	3	(0, 1)	0	1	-2.0	2.0	0.0
7	3	(1, 0)	0	1	-3.0	1.0	0.0
8	3	(1, 1)	1	0	-2.0	2.0	1.0
9	4	(1, 0)	0	1	-3.0	1.0	1.0
10	4	(1, 1)	1	0	-2.0	2.0	2.0
11	5	(0, 1)	0	1	-3.0	2.0	1.0

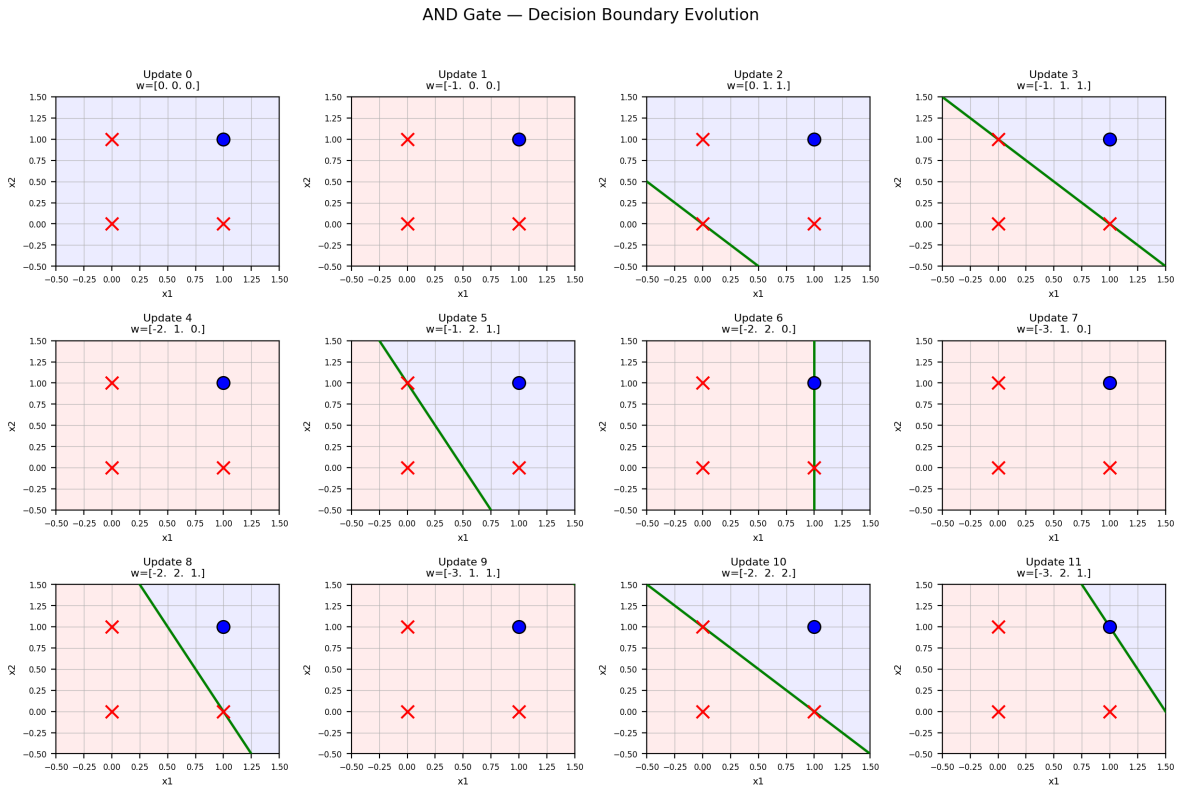

Figure 10: Evolution of the AND gate decision boundary after every weight update. The shaded region shows the class each point in the plane would be assigned (pink = 0, blue = 1); the green line is the boundary $\mathbf{w}^T\mathbf{x} + b = 0$.

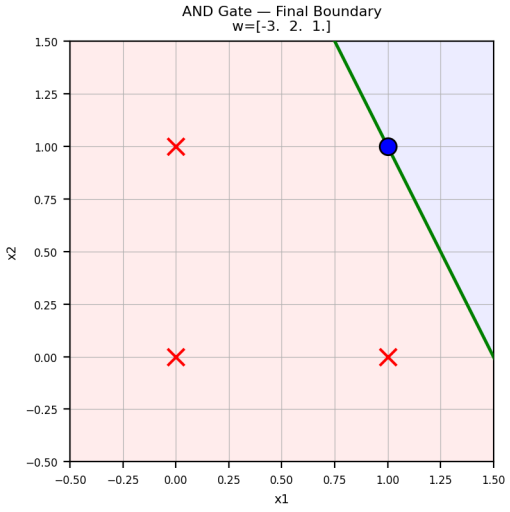

Figure 11: Final converged decision boundary for the AND gate. Note that the point $(1, 1)$ lies exactly on the boundary line, since its net input works out to precisely 0 – it is still correctly classified as 1 because the Step Activation Function uses $z \geq 0 \Rightarrow \hat{y} = 1$.

OR Gate

Table 2: Weight updates during perceptron training for the OR gate.

Update #	Epoch	Input	Target	Prediction	Bias	w1	w2
1	1	(0, 0)	0	1	-1.0	0.0	0.0
2	1	(0, 1)	1	0	0.0	0.0	1.0
3	2	(0, 0)	0	1	-1.0	0.0	1.0
4	2	(1, 0)	1	0	0.0	1.0	1.0
5	3	(0, 0)	0	1	-1.0	1.0	1.0

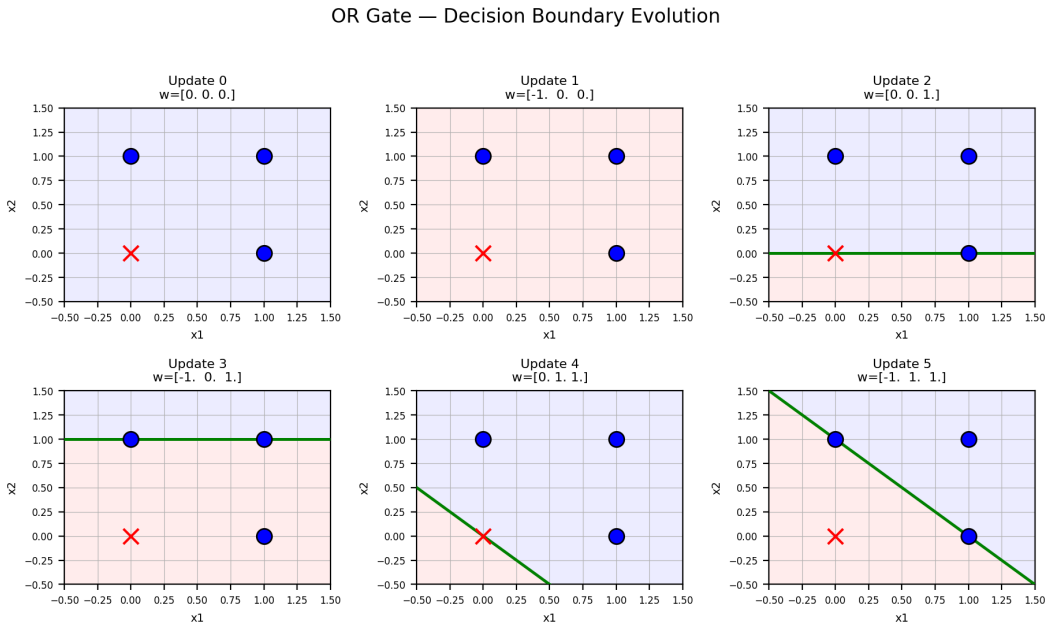

Figure 12: Evolution of the OR gate decision boundary after every weight update.

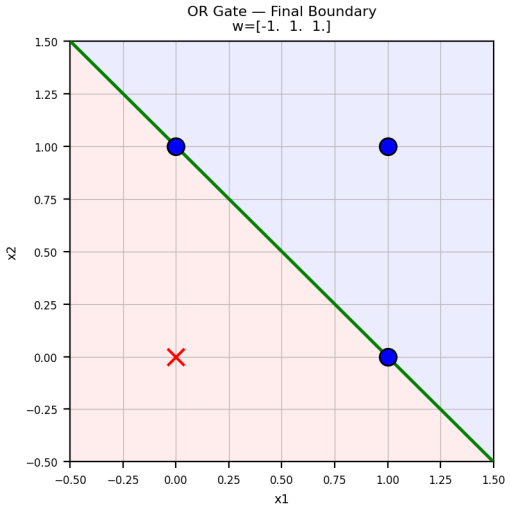

Figure 13: Final converged decision boundary for the OR gate. Here both $(0, 1)$ and $(1, 0)$ lie exactly on the boundary line for the same reason as above.

NOT Gate

Table 3: Weight updates during perceptron training for the NOT gate.

Update #	Epoch	Input	Target	Prediction	Bias	w1
1	1	(1,)	0	1	-1.0	-1.0
2	2	(0,)	1	0	0.0	-1.0

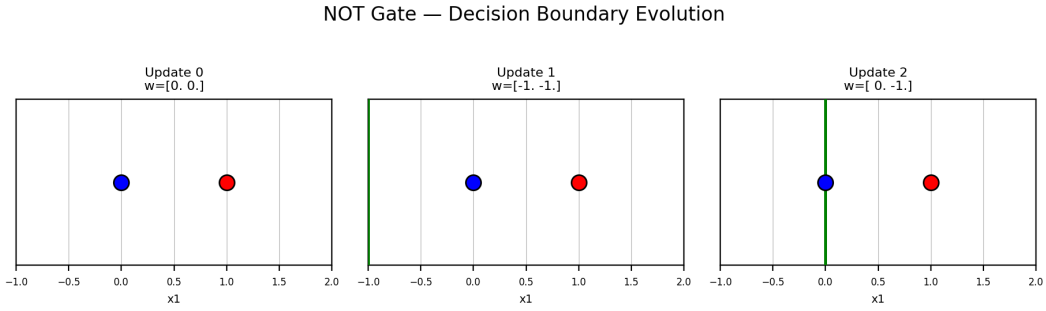

Figure 14: Evolution of the NOT gate decision boundary after every weight update. Since NOT has a single input, the boundary is a vertical threshold line rather than a 2D line.

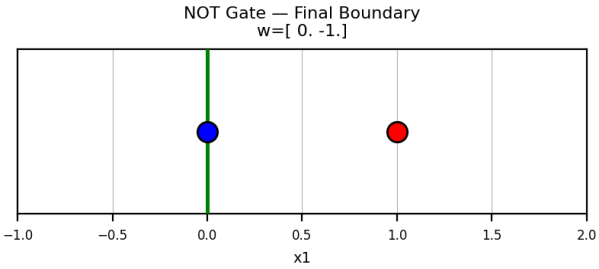

Figure 15: Final converged decision boundary for the NOT gate.

Observations

- Both AND and OR are linearly separable, so the perceptron converges to a zero-error solution within a handful of epochs (11 updates for AND, 5 for OR).
- In both cases the converged boundary line passes exactly through one or more training points. This is expected: the Step Activation Function classifies $z = 0$ as class 1, so any point whose weighted sum lands exactly on 0 is correctly classified while sitting on the boundary itself – this is not an error, simply a property of using integer/binary inputs with a hard 0/1 threshold.
- The NOT gate needs only a single input weight and converges in 2 updates, giving a vertical decision boundary that separates $x_1 = 0$ (output 1) from $x_1 = 1$ (output 0).
- Unlike XOR (discussed in Section 10), all three of these gates are linearly separable, so a single-layer perceptron is sufficient to learn them exactly.

12. Conclusion

A Single Layer Perceptron was implemented from scratch to classify banknotes as authentic or forged using four numerical features. The model achieved a test accuracy of 98.91%, with perfect precision and only three false negatives out of 275 test samples. This shows that the dataset is almost linearly separable, which explains why the perceptron performed so well. At the same time, the training error never became zero, so there is still a small amount of overlap between the two classes. The results were then compared with scikit-learn's Perceptron, and both gave almost the same performance, confirming that the implementation was correct. While trying different settings, it was also observed that when the weights were initialized to zero and a Step activation function was used, changing the learning rate only changed the size of the learned weights and not the final predictions. Feature normalization was also tested. Although it is usually recommended because it makes training more stable, it did not make a noticeable difference for this dataset. Overall, this experiment gave a good understanding of how a perceptron learns and how it behaves on real data. It worked really well for this dataset because the classes are nearly linearly separable, but it also showed its main limitation—it cannot learn non-linear decision boundaries. This is one of the reasons why more advanced models like Multilayer Perceptrons are used for more complex problems.

13. References

1. F. Rosenblatt, "The Perceptron," *Psychological Review*, 1958.
2. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.
3. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
4. S. Haykin, *Neural Networks and Learning Machines*, Pearson, 2009.
5. UCI Machine Learning Repository – Banknote Authentication Dataset.
6. Scikit-learn Documentation: Perceptron.